

Programmation fonctionnelle

C1 Les bases

Dr. Gomis

L3 GL - UFR SET

22 octobre 2025

Plan

Les paradigmes de programmation

La programmation fonctionnelle

Introduction au LISP

Références

Dr. Gomis

Plan

Les paradigmes de programmation

La programmation fonctionnelle

Introduction au LISP

Références

Dr. Gomis

Différents paradigmes

► Programmation impérative

Différents paradigmes

- ▶ Programmation impérative
 - ▶ **procédurale (C, Pascal)**

Différents paradigmes

- ▶ Programmation impérative
 - ▶ procédurale (C, Pascal)
 - ▶ objet (Java, C++, Python)

Différents paradigmes

- ▶ Programmation impérative
 - ▶ procédurale (C, Pascal)
 - ▶ objet (Java, C++, Python)
- ▶ Programmation déclarative

Différents paradigmes

- ▶ Programmation impérative
 - ▶ procédurale (C, Pascal)
 - ▶ objet (Java, C++, Python)
- ▶ Programmation déclarative
 - ▶ fonctionnelle (LISP, Scheme, Haskell, OCaml)

Différents paradigmes

- ▶ Programmation impérative
 - ▶ procédurale (C, Pascal)
 - ▶ objet (Java, C++, Python)
- ▶ Programmation déclarative
 - ▶ fonctionnelle (LISP, Scheme, Haskell, OCaml)
 - ▶ logique (Prolog)

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement
- ▶ Comment faire : On décrit comment résoudre le problème

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement
- ▶ Comment faire : On décrit comment résoudre le problème

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement
- ▶ Comment faire : On décrit comment résoudre le problème

Approche procédurale ou orientée objet

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement
- ▶ Comment faire : On décrit comment résoudre le problème

Approche procédurale ou orientée objet

- ▶ **procédurale** (fonctions, structures, ...)

Dr. Gomis

Programmation impérative

Principe

Dire à l'ordinateur **exactement** quoi faire et dans quel ordre le faire

Caractéristiques principales

- ▶ Séquences d'instructions : "Fais ceci, puis fais cela"
- ▶ Modification d'état : On change la valeur des variables
- ▶ Effets de bord : Les actions modifient l'environnement
- ▶ Comment faire : On décrit comment résoudre le problème

Approche procédurale ou orientée objet

- ▶ procédurale (fonctions, structures, ...)
- ▶ **objet (classes, méthodes, ...)**

Dr. Gomis

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Caractéristiques principales

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Caractéristiques principales

- ▶ On décrit le résultat souhaité, pas les étapes pour y arriver

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Caractéristiques principales

- ▶ On décrit le résultat souhaité, pas les étapes pour y arriver
- ▶ Pas de boucles POUR, TANT QUE, ou de conditions SI explicites dans le code déclaratif pur

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Caractéristiques principales

- ▶ On décrit le résultat souhaité, pas les étapes pour y arriver
- ▶ Pas de boucles POUR, TANT QUE, ou de conditions SI explicites dans le code déclaratif pur
- ▶ Les données ne sont pas modifiées, on crée de nouvelles données (Immutabilité)

Programmation déclarative

Principe

Décrire CE QUE l'on veut, sans spécifier COMMENT l'obtenir

Caractéristiques principales

- ▶ On décrit le résultat souhaité, pas les étapes pour y arriver
- ▶ Pas de boucles POUR, TANT QUE, ou de conditions SI explicites dans le code déclaratif pur
- ▶ Les données ne sont pas modifiées, on crée de nouvelles données (Immutabilité)
- ▶ On définit des relations entre les données plutôt que des séquences d'actions

Dr. Gomis

Programmation déclarative

Un programme déclaratif

Il ne contient pas d'instructions explicites à suivre par l'ordinateur qui l'exécute. Au lieu de cela, le travail de l'ordinateur consiste à manipuler les informations contenues dans le programme afin d'en déduire la solution à un problème donné.

Programmation déclarative

Un programme déclaratif

Il ne contient pas d'instructions explicites à suivre par l'ordinateur qui l'exécute. Au lieu de cela, le travail de l'ordinateur consiste à manipuler les informations contenues dans le programme afin d'en déduire la solution à un problème donné.

Approche fonctionnelle ou logique

Programmation déclarative

Un programme déclaratif

Il ne contient pas d'instructions explicites à suivre par l'ordinateur qui l'exécute. Au lieu de cela, le travail de l'ordinateur consiste à manipuler les informations contenues dans le programme afin d'en déduire la solution à un problème donné.

Approche fonctionnelle ou logique

- Fonctionnelle (fonctions, pas d'instructions, que des expressions, ...)

Programmation déclarative

Un programme déclaratif

Il ne contient pas d'instructions explicites à suivre par l'ordinateur qui l'exécute. Au lieu de cela, le travail de l'ordinateur consiste à manipuler les informations contenues dans le programme afin d'en déduire la solution à un problème donné.

Approche fonctionnelle ou logique

- ▶ Fonctionnelle (fonctions, pas d'instructions, que des expressions, ...)
- ▶ Logique (prédicats, moteurs d'inférence logique)

Dr. Gomis

Plan

Les paradigmes de programmation

La programmation fonctionnelle

Introduction au LISP

Références

Dr. Gomis

Principes fondamentaux

Les fonctions sont des citoyens de première classe

Elles peuvent être traitées comme n'importe quelle autre valeur. On peut :

Principes fondamentaux

Les fonctions sont des citoyens de première classe

Elles peuvent être traitées comme n'importe quelle autre valeur. On peut :

- ▶ Les assigner à des variables

Principes fondamentaux

Les fonctions sont des citoyens de première classe

Elles peuvent être traitées comme n'importe quelle autre valeur. On peut :

- ▶ Les assigner à des variables
- ▶ Les passer en argument à d'autres fonctions (fonctions d'ordre supérieur)

Principes fondamentaux

Les fonctions sont des citoyens de première classe

Elles peuvent être traitées comme n'importe quelle autre valeur. On peut :

- ▶ Les assigner à des variables
- ▶ Les passer en argument à d'autres fonctions (fonctions d'ordre supérieur)
- ▶ Les retourner comme résultat d'autres fonctions.

Principes fondamentaux

Les fonctions sont des citoyens de première classe

Elles peuvent être traitées comme n'importe quelle autre valeur. On peut :

- ▶ Les assigner à des variables
- ▶ Les passer en argument à d'autres fonctions (fonctions d'ordre supérieur)
- ▶ Les retourner comme résultat d'autres fonctions.

Les données sont immuables par défaut

Une fois créée, une structure de données ne sera plus modifiée

Dr. Gomis

Principes fondamentaux

Fonction pure

Principes fondamentaux

Fonction pure

- ▶ Une fonction pure est une fonction dont le résultat dépend uniquement de ses arguments et qui ne produit aucun effet de bord

Principes fondamentaux

Fonction pure

- ▶ Une fonction pure est une fonction dont le résultat dépend uniquement de ses arguments et qui ne produit aucun effet de bord
- ▶ Peu importe le contexte ou le moment où elle est appelée, une fonction pure retourne toujours le même résultat pour les mêmes entrées. Elle n'interagit pas avec des éléments extérieurs car elle ne dépend que de ses arguments

Principes fondamentaux

Fonction pure

- ▶ Une fonction pure est une fonction dont le résultat dépend uniquement de ses arguments et qui ne produit aucun effet de bord
- ▶ Peu importe le contexte ou le moment où elle est appelée, une fonction pure retourne toujours le même résultat pour les mêmes entrées. Elle n'interagit pas avec des éléments extérieurs car elle ne dépend que de ses arguments

Le raisonnement par composition

On construit des programmes complexes en assemblant (composant) de petites fonctions simples et pures.

Dr. Gomis

Principes fondamentaux

Une définition de la PF

La programmation fonctionnelle est un paradigme de programmation qui traite les fonctions comme des citoyens de **première classe**, privilégie l'**immuabilité** des données et l'utilisation de **fonctions pures**, et construit les programmes par **composition de fonctions**.

Principes fondamentaux

Une définition de la PF

La programmation fonctionnelle est un paradigme de programmation qui traite les fonctions comme des citoyens de **première classe**, privilégie l'**immuabilité** des données et l'utilisation de **fonctions pures**, et construit les programmes par **composition de fonctions**.

- Les fonctions peuvent être des valeurs qui sont produites, consommées et composées

Principes fondamentaux

Une définition de la PF

La programmation fonctionnelle est un paradigme de programmation qui traite les fonctions comme des citoyens de **première classe**, privilégie l'**immuabilité** des données et l'utilisation de **fonctions pures**, et construit les programmes par **composition de fonctions**.

- ▶ Les fonctions peuvent être des valeurs qui sont produites, consommées et composées
- ▶ En PF, l'objectif est de maximiser le nombre de fonctions pures, ce qui rend le code plus prévisible, facile à tester et à raisonner

Dr. Gomis

Principes fondamentaux

Une définition de la PF

La programmation fonctionnelle est un paradigme de programmation qui traite les fonctions comme des citoyens de **première classe**, privilégie l'**immuabilité** des données et l'utilisation de **fonctions pures**, et construit les programmes par **composition de fonctions**.

- ▶ Les fonctions peuvent être des valeurs qui sont produites, consommées et composées
- ▶ En PF, l'objectif est de maximiser le nombre de fonctions pures, ce qui rend le code plus prévisible, facile à tester et à raisonner
- ▶ **Calculons la somme des carrés d'une liste de nombres**

Dr. Gomis

En style impératif

```
L = [1, 2, 3, 4, 5]
```

```
s = 0                                # Variable mutable
```

```
for x in L:
    carre = x * x
    s += carre                        # Mutation de l'état
```

```
print(s)
```

En style fonctionnel

$L = [1, 2, 3, 4, 5]$

- On définit de petites fonctions pures et simples

```
def carre(x):  
    return x * x
```

```
def addition(a, b):  
    return a + b
```

En style fonctionnel

- On compose les fonctions pour construire le résultat

```
from functools import reduce  
s = reduce(addition, map(carre, L))  
print(s)
```

Ou plus moderne, avec des "comprehensions"

```
s = sum(x*x for x in L)
```

En style fonctionnel

- ▶ On compose les fonctions pour construire le résultat

```
from functools import reduce  
s = reduce(addition, map(carre, L))  
print(s)
```

Ou plus moderne, avec des "comprehensions"

```
s = sum(x*x for x in L)
```

- ▶ Tout cela devient plus facile dans un langage fonctionnel

Dr. Gomis

Plan

Les paradigmes de programmation

La programmation fonctionnelle

Introduction au LISP

Références

Dr. Gomis

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956
- ▶ Lisp a été conçu pour le " traitement symbolique des données " et tire son nom de l'une de ses principales fonctionnalités : le traitement de listes (LISt Processing)

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956
- ▶ Lisp a été conçu pour le " traitement symbolique des données " et tire son nom de l'une de ses principales fonctionnalités : le traitement de listes (LISt Processing)
- ▶ McCarthy était un chercheur en intelligence artificielle (IA), et bon nombre des fonctionnalités qu'il a intégrées dans sa version initiale du langage en ont fait un excellent langage pour la programmation IA.

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956
- ▶ Lisp a été conçu pour le " traitement symbolique des données " et tire son nom de l'une de ses principales fonctionnalités : le traitement de listes (LISt Processing)
- ▶ McCarthy était un chercheur en intelligence artificielle (IA), et bon nombre des fonctionnalités qu'il a intégrées dans sa version initiale du langage en ont fait un excellent langage pour la programmation IA.
- ▶ Lisp favorise la programmation interactive

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956
- ▶ Lisp a été conçu pour le " traitement symbolique des données " et tire son nom de l'une de ses principales fonctionnalités : le traitement de listes (LISt Processing)
- ▶ McCarthy était un chercheur en intelligence artificielle (IA), et bon nombre des fonctionnalités qu'il a intégrées dans sa version initiale du langage en ont fait un excellent langage pour la programmation IA.
- ▶ Lisp favorise la programmation interactive
- ▶ L'algorithme général d'un interpréteur Lisp se traduit par la boucle : lecture - évaluation - impression

Dr. Gomis

Common Lisp

- ▶ Common Lisp est le descendant moderne du langage Lisp, conçu pour la première fois par *John McCarthy* en 1956
- ▶ Lisp a été conçu pour le " traitement symbolique des données " et tire son nom de l'une de ses principales fonctionnalités : le traitement de listes (LISt Processing)
- ▶ McCarthy était un chercheur en intelligence artificielle (IA), et bon nombre des fonctionnalités qu'il a intégrées dans sa version initiale du langage en ont fait un excellent langage pour la programmation IA.
- ▶ Lisp favorise la programmation interactive
- ▶ L'algorithme général d'un interpréteur Lisp se traduit par la boucle : lecture - évaluation - impression
- ▶ Le Lisp est particulièrement bien adapté à la récursivité.

Dr. Gomis

Un langage fonctionnel

Pour bien programmer en Lisp

Dr. Gomis

Un langage fonctionnel

Pour bien programmer en Lisp

- ▶ Eliminer les affectations pour penser en terme de passage de paramètres,

Un langage fonctionnel

Pour bien programmer en Lisp

- ▶ Eliminer les affectations pour penser en terme de passage de paramètres,
- ▶ Remplacer les itérations par des appels récursifs,

Un langage fonctionnel

Pour bien programmer en Lisp

- ▶ Eliminer les affectations pour penser en terme de passage de paramètres,
- ▶ Remplacer les itérations par des appels récursifs,
- ▶ et la programmation séquentielle en composition de fonctions

Deux sortes d'expressions

Les expressions LISP sont appelées *expressions symboliques* ou *S-expressions*. Elles sont composées de deux objets valides :

► **Un atome**

Deux sortes d'expressions

Les expressions LISP sont appelées *expressions symboliques* ou *S-expressions*. Elles sont composées de deux objets valides :

► Un atome

- symbole : un caractère ou une chaîne de caractères contigus

a t-shirt hello C#77 abc221

Deux sortes d'expressions

Les expressions LISP sont appelées *expressions symboliques* ou *S-expressions*. Elles sont composées de deux objets valides :

► Un atome

- symbole : un caractère ou une chaîne de caractères contigus

a t-shirt hello C#77 abc221

- littéral : un nombre ou une chaîne de caractères (entre " ")

7 21078 "Salut tout le monde" "bon" "b"

Deux sortes d'expressions

Les expressions LISP sont appelées *expressions symboliques* ou *S-expressions*. Elles sont composées de deux objets valides :

► Un atome

- symbole : un caractère ou une chaîne de caractères contigus

a t-shirt hello C#77 abc221

- littéral : un nombre ou une chaîne de caractères (entre " ")

7 21078 "Salut tout le monde" "bon" "b"

- **Une liste** qui est une séquence d'atomes et/ou d'autres listes entre parenthèses.

(ceci est une liste)

Dr. Gomis

Deux symboles particuliers

- **nil** (du latin "rien") encore noté comme une liste vide `()` symbolise en outre le faux (`false`), c'est le seul objet qui soit à la fois une liste et un symbole.

Deux symboles particuliers

- ▶ **nil** (du latin "rien") encore noté comme une liste vide `()` symbolise en outre le faux (`false`), c'est le seul objet qui soit à la fois une liste et un symbole.
- ▶ **t** symbolise le vrai (`true`), il faut d'ailleurs noter que dans un test, tout ce qui n'est pas nil, peut jouer le rôle du vrai

Evaluation des expressions

- Un symbole est évalué comme une variable

Evaluation des expressions

- ▶ Un symbole est évalué comme une variable
- ▶ Une liste est évaluée comme un appel de fonction

(sqrt 2)

Evaluation des expressions

- ▶ Un symbole est évalué comme une variable
- ▶ Une liste est évaluée comme un appel de fonction

(`sqrt` 2)

- ▶ Un littéral est évalué comme elle-même

(+ 7 9)

Evaluation des expressions

- ▶ Un symbole est évalué comme une variable
- ▶ Une liste est évaluée comme un appel de fonction

(`sqrt` 2)

- ▶ Un littéral est évalué comme elle-même

(+ 7 9)

- ▶ L'utilisation d'un seul quote inhibe l'évaluation

'(+ 1 2 3)

Evaluation des expressions

- ▶ Un symbole est évalué comme une variable
- ▶ Une liste est évaluée comme un appel de fonction

(`sqrt` 2)

- ▶ Un littéral est évalué comme elle-même

(+ 7 9)

- ▶ L'utilisation d'un seul quote inhibe l'évaluation

'(+ 1 2 3)

- ▶ Les expressions LISP ne sont pas sensibles à la casse

Dr. Gomis

LISP utilise la notation préfixée

- En notation préfixée, les opérateurs sont écrits avant leurs opérandes.

Ecrivons l'expression suivante en notation préfixée :

$$a * (b + c) / d$$

LISP utilise la notation préfixée

- ▶ En notation préfixée, les opérateurs sont écrits avant leurs opérandes.

Ecrivons l'expression suivante en notation préfixée :

$$a * (b + c) / d$$

- ▶ On obtient l'expression suivante :

`( / ( * a ( + b c ) ) d )`

LISP utilise la notation préfixée

- ▶ En notation préfixée, les opérateurs sont écrits avant leurs opérandes.

Ecrivons l'expression suivante en notation préfixée :

$$a * (b + c) / d$$

- ▶ On obtient l'expression suivante :

$$(/ (* a (+ b c)) d)$$

- ▶ Opérateurs arithmétiques : $+$, $-$, $*$, $/$ et *mod*

LISP utilise la notation préfixée

- ▶ En notation préfixée, les opérateurs sont écrits avant leurs opérandes.

Ecrivons l'expression suivante en notation préfixée :

$$a * (b + c) / d$$

- ▶ On obtient l'expression suivante :

$$(/ (* a (+ b c)) d)$$

- ▶ Opérateurs arithmétiques : $+$, $-$, $*$, $/$ et *mod*
- ▶ Opérateurs de comparaison : $=$ $<$ $<=$ $>$ $>=$

Dr. Gomis

LISP utilise la notation préfixée

- ▶ En notation préfixée, les opérateurs sont écrits avant leurs opérandes.

Ecrivons l'expression suivante en notation préfixée :

$$a * (b + c) / d$$

- ▶ On obtient l'expression suivante :

$$(/ (* a (+ b c)) d)$$

- ▶ Opérateurs arithmétiques : $+$, $-$, $*$, $/$ et *mod*
- ▶ Opérateurs de comparaison : $=$ $<$ $<=$ $>$ $>=$
- ▶ Opérateurs logiques : *and*, *or*, et *not*

Dr. Gomis

Les premières fonctions

L'affectation

Les premières fonctions

L'affectation

► **setq**

```
(setq x 17)
```

Les premières fonctions

L'affectation

► **setq**

```
(setq x 17)
```

► **Setq** permet aussi de grouper des affectations

```
(setq x 1 y (+ 4 5) L '(a b c) M '(d e) S1 's)
```

Les premières fonctions

L'affichage

```
(print x)
```

```
(format nil "~a + ~a = ~a" x y s)
```

```
(format nil "Bonjour ~% chers etudiants")
```

```
(format t "Bonjour") ; affiche la chaine sans " "
```

Les premières fonctions

L'affichage

```
(print x)
```

```
(format nil "~a + ~a = ~a" x y s)  
(format nil "Bonjour ~% chers etudiants")
```

```
(format t "Bonjour") ; affiche la chaine sans " "
```

Les premières fonctions

L'affichage

```
(print x)
```

```
(format nil "~a + ~a = ~a" x y s)
```

```
(format nil "Bonjour ~% chers etudiants")
```

```
(format t "Bonjour") ; affiche la chaine sans " "
```

Lecture

```
(read)
```

```
(setq a (read))
```

Dr. Gomis

Les premières fonctions

La fonction FORMAT utilise deux arguments obligatoires :

- ▶ une destination

Les premières fonctions

La fonction `FORMAT` utilise deux arguments obligatoires :

- ▶ une destination
 - ▶ `T` : envoie le contenu de la chaîne vers la sortie standard ;

Les premières fonctions

La fonction `FORMAT` utilise deux arguments obligatoires :

- ▶ une destination
 - ▶ `T` : envoie le contenu de la chaîne vers la sortie standard ;
 - ▶ `NIL` : génère la chaîne et la rend

Les premières fonctions

La fonction `FORMAT` utilise deux arguments obligatoires :

- ▶ une destination
 - ▶ `T` : envoie le contenu de la chaîne vers la sortie standard ;
 - ▶ `NIL` : génère la chaîne et la rend
- ▶ une chaîne de caractères qui contient du texte et des directives.

Les premières fonctions

La fonction `FORMAT` utilise deux arguments obligatoires :

- ▶ une destination
 - ▶ `T` : envoie le contenu de la chaîne vers la sortie standard ;
 - ▶ `NIL` : génère la chaîne et la rend
- ▶ une chaîne de caractères qui contient du texte et des directives.
- ▶ les autres arguments additionnels fournissent les valeurs que les directives placent dans la chaîne au bon endroit

Les premières fonctions

Pour manipuler une liste

Dr. Gomis

Les premières fonctions

Pour manipuler une liste

- ▶ **car** pour récupérer la tête de la liste

Les premières fonctions

Pour manipuler une liste

- ▶ **car** pour récupérer la tête de la liste
- ▶ **cdr** pour récupérer le reste d'une liste

Les premières fonctions

Pour manipuler une liste

- ▶ **car** pour récupérer la tête de la liste
- ▶ **cdr** pour récupérer le reste d'une liste
- ▶ **nth** pour récupérer le n-ième élément (à partir de 0)

```
(setq L '(1 2 3 4))
```

```
(nth 1 L) ; 2
```

Les premières fonctions

Pour manipuler une liste

- ▶ **car** pour récupérer la tête de la liste
- ▶ **cdr** pour récupérer le reste d'une liste
- ▶ **nth** pour récupérer le n-ième élément (à partir de 0)

```
(setq L '(1 2 3 4))
```

```
(nth 1 L) ; 2
```

- ▶ **setf** pour modifier un élément d'une liste

```
(setf (nth 1 L) "UIDT")
```

Dr. Gomis

Plan

Les paradigmes de programmation

La programmation fonctionnelle

Introduction au LISP

Références

Dr. Gomis

Bibliographie

Tout ouvrage de programmation fonctionnelle

Fin du chapitre

Merci de votre attention